>_ MCMP113

Introduction to Basic Algorithms and Programming

Week 3

Programming Languages, Variables, Data Types and Constants

From algorithm to a running C# program: storing, typing, reading and printing data

```
> dotnet --version
10.0.112
> cd Hello
> dotnet run
Hello, World!

> int age = 19;
> _
```

Asst. Prof. Hasan Oğuz, PhD

Istanbul Okan University · Vocational School

2026–2027 Fall Semester

>_ LEARNING OBJECTIVES

By the End of This Week You Can …

Explain

machine, assembly and high-level languages; how C# is compiled to IL and JIT-compiled by .NET.

Create & run

a console project with dotnet new / dotnet run and understand Program.cs and the .csproj file.

Print

text with Write / WriteLine, escape sequences, interpolation $"…" and format specifiers.

Declare

variables with legal, meaningful names; initialise and assign them; trace their values.

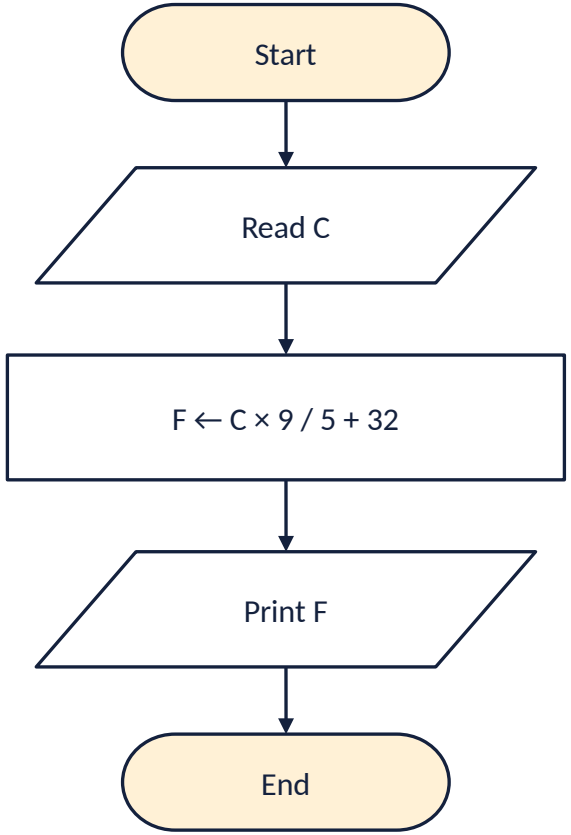
Choose

the right type: int, long, double, decimal, char, string, bool …; use const for fixed values.

Convert

input text with int.Parse / double.Parse, use casts, and recognise overflow and truncation.

From Flowchart to Code: Every Box Becomes a Statement



C# (example 03_05, shortened)

```
// Program.cs
Console.Write("Temperature in °C: ");
double celsius = double.Parse(Console.ReadLine());

double fahrenheit = celsius * 9 / 5 + 32;

Console.WriteLine($"{celsius} °C = {fahrenheit:F1} °F");
```

Today's questions: What is double? Why Parse? What does = do? What does :F1 mean? By the end of the lesson you can answer all four.



PART 1

Programming Languages and .NET

From machine code to C#: how your program is translated and run

>_ THREE LEVELS OF LANGUAGE

Machine Code, Assembly and High-Level Languages

MACHINE CODE

1st generation

```
10111000 00000111
00000000 00000000
00000000
```

Binary instructions the CPU executes directly. Depends on the CPU type (x86, ARM …).

ASSEMBLY

2nd generation

```
mov eax, 7
```

Short names (mnemonics) for machine instructions; an assembler translates them.

HIGH-LEVEL (C#)

3rd generation +

```
int x = 7;
```

Close to human language and maths; runs on many CPUs; a compiler translates it.

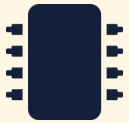
closer to the hardware

closer to people

>_ TRANSLATION

Compiled, Interpreted or Just-in-Time?

Approach	When is the code translated?	Examples	Good	Less good
Compiled (ahead of time)	Before running: the whole program → machine code (.exe)	C, C++, Go, Rust	Very fast start and run	Rebuild for every OS / CPU
Interpreted	While running: statement by statement	Python, shell scripts, classic BASIC	Try code instantly	Usually slower
IL + JIT	Build: C# → IL (.dll). Run: JIT turns IL → machine code	C#, VB.NET (.NET); Java (JVM)	Portable and fast after start-up	Needs the runtime installed



C# belongs to the third row. You compile once (dotnet build); the .NET runtime on each computer finishes the translation at run time (JIT = Just-In-Time).

From Program.cs to the CPU — and SDK vs Runtime



compile time: errors like CS0165 appear here

run time: crashes like FormatException appear here

.NET SDK — for developers

Compiler + dotnet command-line tool + project templates + the runtime.
Install this one on your computer (.NET 10 SDK).

.NET Runtime — for users

Only what is needed to RUN a finished app: the CLR (with the JIT) and the base class library (Console, Math …).

>_ YOUR FIRST PROJECT

Create and Run a Console Project

Terminal (VS Code)

```
> dotnet --version
10.0.112
> dotnet new console -n Hello
The template "Console App" was created successfully.

Processing post-creation actions...
Restoring ...\Hello\Hello.csproj:
Restore succeeded.

> cd Hello
> dotnet run
Hello, World!
```



Open the folder

File ▸ Open Folder… ▸ Hello



Edit Program.cs

Your statements go here



Run

Ctrl+F5, or type dotnet run

Inside the Project: Program.cs and the .csproj File

Hello.csproj (our course template)

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>disable</Nullable>
    <InvariantGlobalization>true</InvariantGlobalization>
  </PropertyGroup>
</Project>
```

Setting	Meaning
Exe	build a runnable program
net10.0	target .NET 10
ImplicitUsings	System etc. imported for us
Nullable disable	no null-warnings on ReadLine()
InvariantGlobalization	72.5 works on Turkish PCs

```
Hello/
   ├─ Hello.csproj  — build settings
   ├─ Program.cs    — your code
   ├─ bin/  obj/    — build output (generated)
```



Without our template, int.Parse(Console.ReadLine()) shows warning CS8604.

Top-Level Statements vs the Classic Main Method

TOP-LEVEL STATEMENTS (C# 9+)

```
// Program.cs  (what we write)
Console.WriteLine("Hello from Main!");
Console.WriteLine("Top-level ...");
```

Console

```
Hello from Main!
The classic form.
```

CLASSIC TEMPLATE (example 03_02)

```
namespace ClassicHello
{
    class Program
    {
        static void Main()
        {
            // Execution starts here.
            Console.WriteLine("Hello from Main!");
            Console.WriteLine("The classic form.");
        }
    }
}
```

Statements, Semicolons, Case and Comments

Program.cs

```
// single-line comment
/* a comment that spans
   several lines */

int age = 19;              // ; ends a statement
Console.WriteLine(age);    // prints 19

// console.WriteLine(age); // CS0103: c ≠ C
// int Age = 20;           // Age ≠ age

int   total   =   5 ;      // spaces ignored ...
int count = 5;             // ... but keep it tidy
```



Semicolon ;

Every statement ends with ; Missing → CS1002



Case-sensitive

Console ≠ console, age ≠ Age



Comments

// line and /* block */ are ignored by the compiler



Whitespace

Free between tokens; indent 4 spaces for people



PART 2

Output, Names and Variables

Printing text, naming things and storing values in memory

>_ OUTPUT

Write vs WriteLine and Escape Sequences

What is printed?

```
Console.Write("A");
Console.Write("B");
Console.WriteLine("C");
Console.WriteLine("D");
```

Console

```
ABC
D
```

`Write` keeps the cursor on the line. **`WriteLine`** ends the line.

Escape	Meaning	Example in code	Printed
`\n`	new line	`"Line 1\nLine 2"`	`Line 1 ⏎ Line 2`
`\t`	tab	`"Name:\tAyşe"`	`Name:  Ayşe`
`\"`	double quote	`"Say \"hi\""`	`Say "hi"`
`\\`	backslash	`"C:\\MCMP113"`	`C:\MCMP113`
`\'`	single quote	`'\''  (a char)`	`'`
`@"..."`	verbatim string	`@"C:\MCMP113"`	`C:\MCMP113`



"C:\Temp\new" → error CS1009: Unrecognized escape sequence

Worked Example: A Personal Student Card

code/03_01_HelloCard/Program.cs

```
Console.WriteLine("Hello, Okan!");

Console.Write("Enter your name: ");     // cursor stays on this line
string name = Console.ReadLine();
Console.Write("Enter your programme: ");
string programme = Console.ReadLine();

Console.WriteLine();                    // an empty line
Console.WriteLine("===== STUDENT CARD =====");
Console.WriteLine($"Name:\t{name}");    // \t = tab
Console.WriteLine($"Prog.:\t{programme}");
Console.WriteLine("Motto:\t\"Code, test, repeat.\"");  // \" = "
Console.WriteLine("Folder:\tC:\\MCMP113\\Week03");     // \\ = \
Console.WriteLine(@"Same:   C:\MCMP113\Week03");  // verbatim
Console.Write("Line 1\nLine 2\n");                 // \n = new line
```

Console

```
Hello, Okan!
Enter your name: Ayşe Yılmaz
Enter your programme:
Computer Programming

===== STUDENT CARD =====
Name:   Ayşe Yılmaz
Prog.:  Computer Programming
Motto:  "Code, test, repeat."
Folder: C:\MCMP113\Week03
Same:   C:\MCMP113\Week03
Line 1
Line 2
```

Naming Rules and Naming Conventions

Name	OK?	Why
`studentName`	✔	letters only, camelCase
`total2`	✔	digits allowed after the first character
`_count`	✔	_ is allowed (even at the start)
`2ndPlace`	✘	starts with a digit
`first name`	✘	contains a space
`price-tl`	✘	- is the minus operator
`class`	✘	reserved keyword
`öğrenciAdı`	✔	legal, but use English names

Conventions (style)

camelCase

local variables

`examGrade, unitPrice`

PascalCase

constants, types, methods

`VatRate, Console, WriteLine`

Meaningful

say what it stores

`x → examGrade`
`n → studentCount`

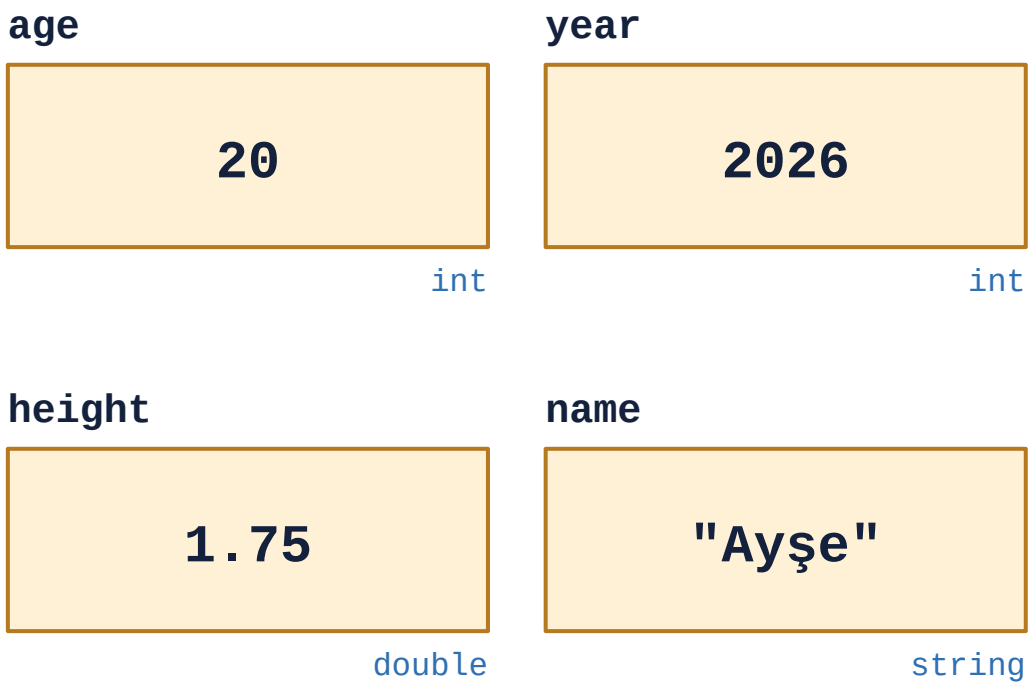
A Variable Is a Named Box in Memory

Four statements

```
int age;            // 1 declaration
age = 19;           // 2 assignment
int year = 2026;    // 3 declare + initialise
age = age + 1;      // 4 read, add, store back
```

After line	age	year
1	–	–
2	19	–
3	19	2026
4	20	2026

Memory (RAM) after line 4 — plus two more variables



= means "store into" (←)

Trace It Yourself — and the Unassigned Variable Error

Trace me

```
int x = 5;
int y = x * 2;
x = x + 3;
y = y - x;
Console.WriteLine($"x = {x}, y = {y}");
```

Line	x	y	Output
1	5	–	
2	5	10	
3	8	10	
4	8	2	
5	8	2	**x = 8, y = 2**

Does not compile

```
int total;
Console.WriteLine(total);
```

Build output

```
error CS0165: Use of unassigned local
variable 'total'
```

```
int x = 5;
int x = 6;      // CS0128
```



Fix: `int total = 0;` `x = 6;`

Worked Example: Swapping Two Variables

code/03_08_SwapVariables/Program.cs

```
Console.Write("a = ");
int a = int.Parse(Console.ReadLine());
Console.Write("b = ");
int b = int.Parse(Console.ReadLine());
Console.WriteLine($"Before: a = {a}, b = {b}");

int temp = a;    // 1) save a copy of a
a = b;           // 2) overwrite a with b
b = temp;        // 3) b gets the saved copy

Console.WriteLine($"After : a = {a}, b = {b}");
```

Console

```
a = 5
b = 9
Before: a = 5, b = 9
After : a = 9, b = 5
```

Statement	a	b	temp
(input)	5	9	–
int temp = a;	5	9	5
a = b;	9	9	5
b = temp;	9	**5**	5

Wrong: without temp

```
a = b;   // a = 9, the 5 is lost
b = a;   // b = 9
```

Result: a = 9, b = 9



PART 3

Data Types and Constants

Choosing the right box: integers, real numbers, money, text and truth values

The Most Important C# Data Types

Type	Bytes	Range / precision	Literal	Typical use
`byte`	1	0 … 255	`200`	small counts, raw data
`short`	2	−32 768 … 32 767	`30000`	rarely needed
`int`	4	−2 147 483 648 … 2 147 483 647	`42` `1_000_000`	default whole number
`long`	8	≈ $\pm 9.2 \times 10^{18}$	`42L`	very large whole numbers
`float`	4	≈ $\pm 3.4 \times 10^{38}$, ~6–9 digits	`3.14f`	graphics, sensors
`double`	8	≈ $\pm 1.7 \times 10^{308}$, ~15–17 digits	`3.14` `2.5e3`	default real number
`decimal`	16	≈ $\pm 7.9 \times 10^{28}$, 28–29 digits	`19.99m`	money
`char`	2	one character (U+0000 … U+FFFF)	`'A'`	a letter, a grade
`bool`	1	true or false	`true`	yes / no
`string`	varies	any text (sequence of chars)	`"Ayşe"`	names, IDs, phones

float, double or decimal? Precision Matters

```
Same sums, three types
Console.WriteLine(0.1 + 0.2);     // double
Console.WriteLine(0.1f + 0.2f);   // float
Console.WriteLine(0.1m + 0.2m);   // decimal

Console.WriteLine(1.0 / 3);
Console.WriteLine(1.0f / 3);
Console.WriteLine(1.0m / 3);
```

Console

```
0.30000000000000004
0.3
0.3

0.3333333333333333
0.33333334
0.3333333333333333333333333333
```



double

Measurements, science, maths. Our default for real numbers.



decimal

Money: prices, salaries, VAT. Exact in base 10.



float

Only when memory matters (graphics, sensors). Needs f.

>_ DESIGN DECISION

Which Type Would You Choose?



Number of students

`int`



World population

`long`



Body temperature

`double`



Product price

`decimal`



Passed the course?

`bool`



Letter grade B

`char`



Student name

`string`



T.C. Kimlik No, phone

`string`



Rule of thumb: if you never calculate with it, it is not a number — store it as a string.

Asking C# for the Limits — and the var Keyword

dotnet run (03_10_TypeLimits)

```
type      bytes  MinValue                                MaxValue
byte          1  0                                       255
short         2  -32768                                  32767
int           4  -2147483648                             2147483647
long          8  -9223372036854775808                    9223372036854775807
float         4  -3.4028235E+38                          3.4028235E+38
double        8  -1.7976931348623157E+308                1.7976931348623157E+308
decimal      16  -79228162514264337593543950335          79228162514264337593543950335
char          2  0                                       65535
bool          1  False                                   True
```

```
var a = 42;      // int
var e = 4.2m;    // decimal
```

var: inferred types

```
42 -> Int32, 42L -> Int64, 4.2 -> Double
4.2f -> Single, 4.2m -> Decimal, 'x' -> Char, "x" -> String
```

Constants: Values That Must Never Change

code/03_03_CircleArea/Program.cs

```
const double Pi = 3.14159;      // our constant: can never change

Console.Write("Radius (cm): ");
double radius = double.Parse(Console.ReadLine());

double area = Pi * radius * radius;          // A = πr²
double circumference = 2 * Pi * radius;      // C = 2πr
double areaExact = Math.PI * radius * radius;

Console.WriteLine($"Area          : {area:F4} cm²");
Console.WriteLine($"Circumference : {circumference:F4} cm");
Console.WriteLine($"Area (Math.PI): {areaExact:F4} cm²");
```

Console

```
Radius (cm): 2.5
Area          : 19.6349 cm²
Circumference : 15.7080 cm
Area (Math.PI): 19.6350 cm²
```

const = locked

const double Pi = 3.14159;
Pi = 3.0; → error CS0131

Why use them?

No magic numbers (VatRate, not 0.20m). Change in one place. PascalCase names.

Worked Example: A Receipt with decimal and KDV

code/03_07_Receipt/Program.cs

```
const decimal VatRate = 0.20m;      // KDV 20 %  (m = decimal)

Console.Write("Product name        : ");
string product = Console.ReadLine();
Console.Write("Unit price (TL, net): ");
decimal unitPrice = decimal.Parse(Console.ReadLine());
Console.Write("Quantity            : ");
int quantity = int.Parse(Console.ReadLine());

decimal subtotal = unitPrice * quantity;  // int -> decimal
decimal vat = subtotal * VatRate;
decimal total = subtotal + vat;

Console.WriteLine("--------------------------------");
Console.WriteLine($"{product,-12} {quantity,3} x {unitPrice,8:F2}");
Console.WriteLine($"{"Subtotal",-18} {subtotal,10:F2} TL");
Console.WriteLine($"{"KDV 20%",-18} {vat,10:F2} TL");
Console.WriteLine($"{"TOTAL",-18} {total,10:F2} TL");
```

Console

```
Product name        : Notebook
Unit price (TL, net): 24.90
Quantity            : 3
--------------------------------
Notebook       3 x    24.90
Subtotal                74.70 TL
KDV 20%                 14.94 TL
TOTAL                   89.64 TL
```

Pseudocode

```
READ product, unitPrice, quantity
subtotal ← unitPrice × quantity
vat ← subtotal × 0.20
total ← subtotal + vat
PRINT subtotal, vat, total
```

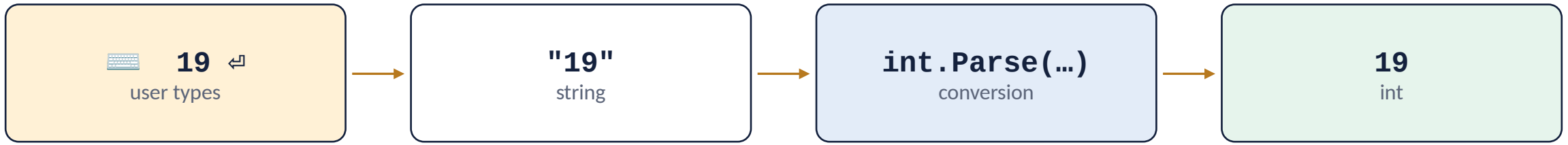


PART 4

Input and Type Conversion

Reading text from the keyboard, converting it, and the traps on the way

Console.ReadLine() Always Returns a string



code/03_12_InputParsing/Program.cs (lines 3–12)

```
Console.Write("Your age: ");
string ageText = Console.ReadLine();   // text, e.g. "19"
int age = int.Parse(ageText);          // number 19

Console.WriteLine($"Text   + 1 = {ageText + 1}");  // joins text
Console.WriteLine($"Number + 1 = {age + 1}");      // adds

Console.Write("Height in m (e.g. 1.75): ");
double height = double.Parse(Console.ReadLine());
Console.WriteLine($"Height in cm: {height * 100}");
```

Console

```
Your age: 19
Text   + 1 = 191
Number + 1 = 20
Height in m (e.g. 1.75):
1.75
Height in cm: 175
```

Parse, Convert and TryParse

Input text	int.Parse(text)	Convert.ToInt32(text)	int.TryParse(text, out int n)
"42"	42	42	true, n = 42
" 42 "	42	42	true, n = 42
"3.5"	FormatException	FormatException	false, n = 0
"abc" or ""	FormatException	FormatException	false, n = 0
"99999999999"	OverflowException	OverflowException	false, n = 0

Crash: letters typed instead of digits (abridged)

```
Your age: nineteen
Unhandled exception. System.FormatException: The input string 'nineteen' was not in a correct format.
   at System.Int32.Parse(String s)
   at Program.<Main>$(String[] args) in ...\03_12_InputParsing\Program.cs:line 5
```

72.5 or 72,5? The Turkish Decimal Comma Trap

Typed	Turkish PC, default project	Our template (invariant)
72.5	725 ✗ wrong	72.5 ✓
72,5	72.5 ✓	725 ✗ wrong
output of 72.5	72,5	72.5

Real run of 03_06_Bmi with a comma

```
Weight (kg): 72,5
Height (m) : 1.78
BMI = 228.8 kg/m²
```

```
using System.Globalization;   // alternative fix
double w = double.Parse(text, CultureInfo.InvariantCulture);
```

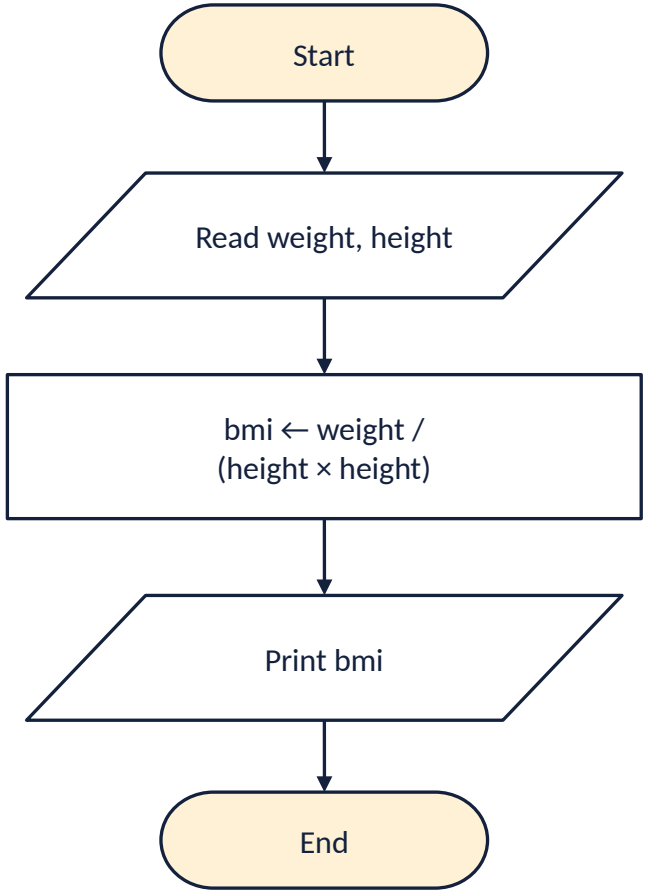
Course rule

Always type a decimal POINT: 72.5, 1.78, 24.90.

Why so dangerous?

No error message! The program runs and prints a wrong number. Always sanity-check results.

Worked Example: Body Mass Index



code/03_06_Bmi/Program.cs

```
Console.Write("Weight (kg): ");
double weight = double.Parse(Console.ReadLine());
Console.Write("Height (m) : ");
double height = double.Parse(Console.ReadLine());

// parentheses: divide by the SQUARE of the height
double bmi = weight / (height * height);

Console.WriteLine($"BMI = {bmi:F1} kg/m²");
```

Console

```
Weight (kg): 72.5
Height (m) : 1.78
BMI = 22.9 kg/m²
```

Check by hand

1.78 × 1.78 = 3.1684

72.5 / 3.1684 ≈ 22.88

Worked Example: Celsius to Fahrenheit

code/03_05_CelsiusToFahrenheit/Program.cs

```
Console.Write("Temperature in °C: ");
double celsius = double.Parse(Console.ReadLine());

// celsius is a double, so celsius * 9 / 5 is computed as double
double fahrenheit = celsius * 9 / 5 + 32;

Console.WriteLine($"{celsius} °C = {fahrenheit:F1} °F");
```

three runs

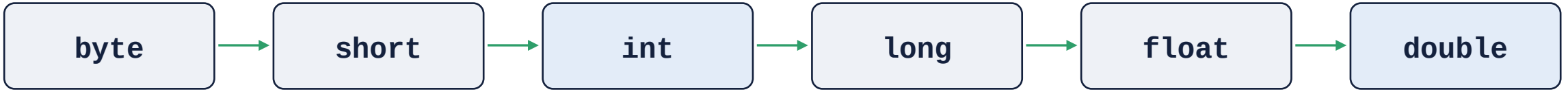
```
Temperature in °C: 36.6
36.6 °C = 97.9 °F
Temperature in °C: 100
100 °C = 212.0 °F
Temperature in °C: -40
-40 °C = -40.0 °F
```

Trap: integer division

```
double c = 20;
Console.WriteLine(9 / 5 * c + 32);  // 52  ✗
Console.WriteLine(c * 9 / 5 + 32);  // 68  ✓
```

9 / 5 is computed with two ints → 1, not 1.8. Write 9.0 / 5 or start with the double variable.

Implicit and Explicit Conversions



implicit (automatic, widening) →

← explicit cast needed: (int)x

code/03_11_CastDemo/Program.cs (lines 8–23, blank lines removed)

```
double price = 3.99;
int truncated = (int)price;            // cast: fraction CUT OFF
int rounded = Convert.ToInt32(price);  // Convert: rounds
Console.WriteLine($"(int){price} = {truncated}");
Console.WriteLine($"Convert.ToInt32({price}) = {rounded}");
double negative = -3.99;
Console.WriteLine($"(int){negative} = {(int)negative}");
double precise = 1.0 / 3;
float lessPrecise = (float)precise;    // fewer digits kept
Console.WriteLine($"double {precise} -> float {lessPrecise}");
int big = 300;
byte small = (byte)big;    // 300 does not fit in 0..255
Console.WriteLine($"(byte){big} = {small}");
```

dotnet run (03_11_CastDemo)

```
int 7 -> long 7 -> double 7
(int)3.99 = 3
Convert.ToInt32(3.99) = 4
(int)-3.99 = -3
double 0.3333333333333333
  -> float 0.33333334
(byte)300 = 44
2147483647 + 1 = -2147483648
```

Overflow: When the Box Is Too Small

code/03_13_OverflowChecked/Program.cs

```
Console.Write("Start value: ");
int start = int.Parse(Console.ReadLine());
Console.Write("Add        : ");
int step = int.Parse(Console.ReadLine());

int silent = start + step;             // may wrap around
Console.WriteLine($"int     : {silent}");

long exact = (long)start + step;       // long has room
Console.WriteLine($"long    : {exact}");

int safe = checked(start + step);      // throws on overflow
Console.WriteLine($"checked : {safe}");
```



`int big = int.MaxValue + 1;` → compile error CS0220 (constants are checked)

run 1: fits

```
Start value: 100
Add        : 23
int     : 123
long    : 123
checked : 123
```

run 2: too big for int

```
Start value: 2000000000
Add        : 500000000
int     : -1794967296
long    : 2500000000
Unhandled exception.
System.OverflowException: Arithmetic
operation resulted in an overflow.
```

String Interpolation and Format Specifiers

In code	Value	Printed	Meaning
`{x}`	`1.0 / 3`	`0.3333333333333333`	default: all significant digits
`{x:F2}`	`3.14159`	`3.14`	fixed, 2 decimals
`{n:N0}`	`1234567`	`1,234,567`	thousands separator, 0 decimals
`{x:N2}`	`1234.5`	`1,234.50`	separator + 2 decimals
`{h:D2}`	`7`	`07`	integer, at least 2 digits
`[{n,5}]`	`42`	`[   42]`	width 5, right-aligned
`[{n,-5}]`	`42`	`[42   ]`	width 5, left-aligned
`{p:P0}`	`0.2`	`20 %`	percent
`{d:C}`	`1234.5m`	`¤1,234.50`	currency (invariant ¤) — avoid

Money in this course: `$"{total:F2} TL"`

Worked Example: Seconds → Hours, Minutes, Seconds

code/03_09_SecondsToHms/Program.cs

```
Console.Write("Total seconds: ");
int totalSeconds = int.Parse(Console.ReadLine());

int hours = totalSeconds / 3600;         // whole hours
int minutes = totalSeconds % 3600 / 60;  // leftover minutes
int seconds = totalSeconds % 60;         // leftover seconds

Console.WriteLine($"{hours} h {minutes} min {seconds} s");
Console.WriteLine($"Clock: {hours:D2}:{minutes:D2}:{seconds:D2}");
```

two runs

```
Total seconds: 3725
1 h 2 min 5 s
Clock: 01:02:05
Total seconds: 86399
23 h 59 min 59 s
Clock: 23:59:59
```

Pseudocode

```
READ total
h ← total div 3600
m ← (total mod 3600) div 60
s ← total mod 60
PRINT h, m, s
```

Expression	total = 3725
total / 3600	1
total % 3600	125
125 / 60	2
total % 60	5

Compiler Errors You Will Meet This Week

You wrote	Error	Fix
`int age = Console.ReadLine();`	CS0029: Cannot implicitly convert type 'string' to 'int'	`int.Parse(Console.ReadLine())`
`decimal price = 19.99;`	CS0664: Literal of type double … use an 'M' suffix	`19.99m`
`float f = 3.14;`	CS0664: … use an 'F' suffix	`3.14f`
`int n = 3.9;`	CS0266: Cannot implicitly convert type 'double' to 'int'	`(int)3.9` — if you want 3
`int total;` …`(total)`	CS0165: Use of unassigned local variable 'total'	`int total = 0;`
`console.WriteLine(x);`	CS0103: The name 'console' does not exist …	`Console` (capital C)
`Console.WriteLine(x)`	CS1002: ; expected	add `;`
`Pi = 3.0;` (Pi is const)	CS0131: The left-hand side of an assignment must be a variable …	do not change constants

Silent Bugs: It Compiles, but the Answer Is Wrong

Text + number = joining

`"Total: " + 2 + 3` → `Total: 23`

Use parentheses: "Total: " + (2 + 3) → Total: 5

Integer division

`9 / 5 * 20 + 32` → `52`

Use 9.0 / 5 → 68

Comma instead of point

`double.Parse("72,5")` → `725`

Type 72.5 (course rule)

Overflow wraps around

`int.MaxValue + 1  (variable)` → `-2147483648`

Use long or checked(...)

Quiz 1: What Does This Program Print?

Predict all six lines

```
int a = 7;
int b = a;
a = 10;
Console.WriteLine($"{a} {b}");   // (1)
Console.WriteLine(5 / 2);          // (2)
Console.WriteLine(5 / 2.0);        // (3)
Console.WriteLine("5" + 2);        // (4)
Console.WriteLine(1 + 2 + "3");    // (5)
Console.WriteLine((int)9.99);      // (6)
```



Rules

Work alone, on paper, 3 minutes. No computer!



Hint

Ask for every line: which TYPES are involved — int, double or string?

Quiz 1: Answers

Real output

```
10 7
2
2.5
52
33
9
```

#	Why
1	b = a copies the VALUE 7; later a = 10 does not affect b
2	int / int → int: 2 (fraction dropped)
3	one operand is double → 2.5
4	string + number → text is joined: "52"
5	left to right: 1 + 2 = 3, then 3 + "3" = "33"
6	cast truncates: (int)9.99 = 9

Quiz 2: Find All the Bugs

buggy.cs

```
1  Console.Write("Price per kg: ")
2  decimal price = Console.ReadLine();
3  const decimal Vat = 0.20;
4  int quantity = 2.5;
5  decimal Total = price * quantity * (1 + vat);
6  console.WriteLine(total);
```

Goal: read the price per kg, buy 2.5 kg, add 20 % KDV, print the total. Find all seven errors on the six lines.

Work in pairs

For each line: which rule is broken?

- statement syntax
- string vs number
- literal suffixes
- int vs double
- case sensitivity

Quiz 2: Answers and the Corrected Program

Line	Error	Fix
1	CS1002 ; expected	`add ;`
2	CS0029 string → decimal	`decimal.Parse(Console.ReadLine())`
3	CS0664 double literal	`0.20m`
4	CS0266 double → int	`decimal quantity = 2.5m;`
5	CS0103 vat ≠ Vat	`(1 + Vat), decimal total`
6	CS0103 console, total	`Console.WriteLine(total)`



The first build shows only CS1002: a syntax error hides the other errors. Fix, rebuild, repeat.

fixed.cs

```
Console.Write("Price per kg: ");
decimal price =
    decimal.Parse(Console.ReadLine());
const decimal Vat = 0.20m;
decimal quantity = 2.5m;
decimal total =
    price * quantity * (1 + Vat);
Console.WriteLine($"{total:F2}");
```

Console

```
Price per kg: 100
300.00
```

>_ IN-CLASS EXERCISES · 25 MIN

Your Turn: In-Class Exercises

E1 · Choose types

Pick a type for: number of pages, fuel price, is a member?, room code "B207", distance in km, initial of your name.

E2 · Age calculator

Read name and birth year; print "Ayşe, in 2026 you turn 19." Use const int CurrentYear = 2026;

E3 · km → miles

Read a distance in km; print miles with 3 decimals. 1 mile = 1.609344 km (const).

E4 · Average of three

Read three grades (double); print the average with F2. Remember the Week 1 parentheses bug!

E5 · Back to seconds

The reverse of 03_09: read hours, minutes and seconds; print the total in seconds (1, 23, 20 → 5000).

E6 · Swap trace

Trace x = 3, y = 8: t = x; x = y; y = t; — then try to swap without t using + and −.

>_ HOMEWORK · BEFORE WEEK 4

Homework

#	Task	Practises
H1	Cylinder volume V = πr²h (read r and h, use Math.PI, print F2)	double, Math.PI, F2
H2	Currency converter: TL → EUR with the rate typed by the user	decimal, Parse
H3	Minutes → days, hours, minutes (e.g. 100000 min)	int, / and %
H4	short s = short.MaxValue; s = (short)(s + 1); — predict, run, explain the output	overflow, casts
H5	Read the notes §1–§7 and do 3 exercises from "Extra practice"	everything



Every program: header comment, meaningful names, prompts with Write, output with F2 where needed, tested with 2–3 inputs.

>_ SUMMARY

Key Takeaways

01 C# is compiled to IL (dotnet build); the .NET runtime JIT-compiles IL to machine code.

02 A variable = name + type + value. = stores a value (←); read before assign → CS0165.

03 Default types: int for whole numbers, double for measurements, decimal for money, string for text/IDs.

04 const protects fixed values; Math.PI is built in.

05 ReadLine() returns a string → convert with int.Parse / double.Parse; type a decimal POINT.

06 Casts truncate, int overflows silently, 9 / 5 == 1 — test with known answers.

NEXT WEEK

Week 4: Arithmetic, Relational and Logical Operators